

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)**Department of Computer Science and Engineering****ASSESSMENT TOOL 2 – Pseudocode Writing Task**

Course Code:	CSA0615	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO5 – Naturalization (BL5, BL6)	Assessment:	Assessment Tool 2 – Pseudocode Writing Task
Weightage:	30% of CIA	Total Marks:	100
CO5 Statement:	<i>Analyze computational problems using backtracking techniques and complexity classes such as P, NP, NP-Hard, and NP-Complete for combinatorial problem solving.</i>		
Student Name:	SOUNDARIYA M	Reg. No.:	192524342
Section / Batch:	B.TECH AI&DS	Date:	31-08-2026

Instructions to Students

- Answer ALL questions with proper pseudocode and logical explanation.
- Clearly specify the algorithmic approach and complexity class wherever applicable.
- Use proper asymptotic notation (Big-O, Big- Θ , Big- Ω) for complexity analysis.
- Include state-space representation, recursion steps, and pruning conditions in backtracking problems.
- Neat presentation and step-by-step justification are mandatory.

Question Type	Focus Area	Questions	Marks
Pseudocode Writing Task	Analyze combinatorial problems using backtracking and complexity classes such as P, NP, NP-Hard, and NP Complete.	Q1 – Q2	Q1 –50 Q2 –50
GRAND TOTAL:		100 Marks	

SECTION A – Pseudocode Writing Task

Graph Coloring Problem with M Colors

Develop a pseudocode using Backtracking to color the vertices of a graph using at most m colors such that adjacent vertices have different colors. Clearly identify inputs (graph, number of colors m) and output (valid coloring or failure). Design a logically correct algorithm that assigns colors vertex by vertex while checking adjacency constraints. Use loops, recursive calls, and conditional validation effectively during the coloring process. Apply pruning to eliminate invalid color assignments early. Ensure the pseudocode is clear, well-structured, and easy to follow. Handle all possible cases and guarantee correctness of the coloring result.

Q1 – GRAPH COLORING PROBLEM WITH M COLORS

Backtracking Approach

1. Problem Statement

Given a graph $G = (V, E)$ and an integer m , color all vertices of the graph using **at most m colors** such that **no two adjacent vertices have the same color**.

The problem is solved using **Backtracking**. A color is assigned to one vertex at a time. Before assigning a color, the algorithm checks whether that color is safe with respect to all already-colored adjacent vertices. If no color is possible, the algorithm backtracks and changes the color of a previously colored vertex.

2. Input and Output

Input

- $G \rightarrow$ Graph with V vertices.
- $V \rightarrow$ Number of vertices.
- $m \rightarrow$ Maximum number of colors available.

- $G[i][j] = 1$ if an edge exists between vertices i and j ; otherwise 0.

Output

- A valid coloring of all vertices using at most m colors, if possible.
 - "No valid coloring exists" if the graph cannot be colored using m colors.
-

3. Pseudocode

ALGORITHM GRAPH_COLORING(G, V, m)

INPUT:

$G \rightarrow$ Graph with V vertices

$V \rightarrow$ Number of vertices

$m \rightarrow$ Maximum number of colors

OUTPUT:

A valid coloring of all vertices

OR report failure

BEGIN

color[1...V] $\leftarrow$ 0

IF COLOR_GRAPH(1) = TRUE THEN

PRINT "Valid Coloring:"

FOR $i \leftarrow 1$ TO V DO

PRINT "Vertex", i , " $\rightarrow$ Color", color[i]

END FOR

RETURN

ELSE

PRINT "No valid coloring exists"

END IF

END

FUNCTION COLOR_GRAPH(vertex)

BEGIN

IF vertex > V THEN

 RETURN TRUE

END IF

FOR $c \leftarrow 1$ TO m DO

 IF IS_SAFE(vertex, c) = TRUE THEN

 color[vertex] $\leftarrow$ c

 IF COLOR_GRAPH(vertex + 1) = TRUE THEN

RETURN TRUE

END IF

color[vertex] $\leftarrow$ 0

END IF

END FOR

RETURN FALSE

END

FUNCTION IS_SAFE(vertex, c)

BEGIN

FOR $i \leftarrow 1$ TO V DO

IF $G[\text{vertex}][i] = 1$ AND $\text{color}[i] = c$ THEN

RETURN FALSE

END IF

END FOR

RETURN TRUE

END

4. Logical Explanation

The algorithm starts with all vertices uncolored.

1. Select the first vertex.
2. Try colors from 1 to m.
3. Before assigning a color, check all adjacent vertices.
4. If no adjacent vertex has the same color, assign the color.
5. Recursively move to the next vertex.
6. If all vertices are successfully colored, return TRUE.
7. If no color can be assigned to the current vertex, undo the previous assignment.
8. Try another color for the previous vertex.
9. Continue this process until a valid coloring is found or all possibilities fail.

Thus, the algorithm systematically explores possible color assignments while eliminating invalid assignments early.

5. Example

Consider a graph with four vertices:

Vertices = {A, B, C, D}

Edges:

A-B

A-C

B-C

C-D

Let:

$$m = 3$$

One possible coloring is:

Vertex	Color
A	1
B	2
C	3
D	1

Adjacent vertices have different colors.

Therefore:

Valid Coloring:

$A \rightarrow 1$

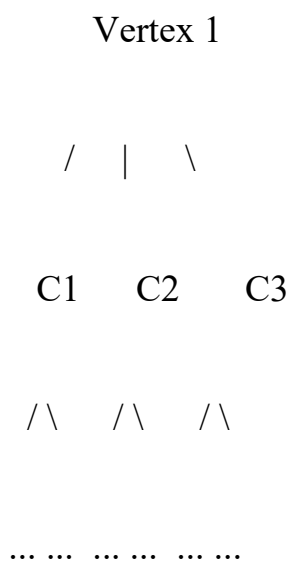
$B \rightarrow 2$

$C \rightarrow 3$

$D \rightarrow 1$

6. State-Space Representation

The state-space tree represents different color choices for each vertex.



- **Level 1** $\rightarrow$ Color assigned to vertex 1

- **Level 2** $\rightarrow$ Color assigned to vertex 2
- **Level 3** $\rightarrow$ Color assigned to vertex 3
- ...
- **Level V** $\rightarrow$ Color assigned to vertex V

Each node represents a **partial coloring**.

A branch is expanded only when the selected color does not violate the adjacency constraint.

7. Recursion Steps

1. Start with vertex 1.
2. Try color 1.
3. Check whether color 1 is safe.
4. If safe, assign it and recursively process the next vertex.
5. Try the remaining colors when necessary.
6. Continue until all vertices are colored.
7. If a vertex cannot receive any color, backtrack.

8. Remove its previous color assignment.

9. Try another color.

10. If every possibility fails, report failure.

8. Pruning Conditions

Invalid branches are eliminated immediately.

Condition 1 – Adjacent vertex has the same color

IF $G[\text{vertex}][i] = 1$ AND $\text{color}[i] = c$

RETURN FALSE

This prevents adjacent vertices from receiving the same color.

Condition 2 – No available color

If all m colors violate the constraints:

RETURN FALSE

The algorithm backtracks to the previous vertex.

These conditions prevent unnecessary exploration of invalid color combinations.

9. Correctness

The algorithm is correct because:

1. Every vertex is considered for coloring.
2. Only colors from 1 to m are used.
3. A color is assigned only after checking all adjacent vertices.
4. Therefore, no two adjacent vertices receive the same color.
5. If all V vertices are colored, the resulting assignment is a valid m -coloring.
6. If a partial coloring cannot be completed, backtracking removes the last assignment and tries another possibility.
7. Since all possible valid color assignments are systematically considered, the algorithm finds a valid coloring whenever one exists.

Hence, the algorithm correctly determines whether the graph can be colored using at most m colors

10. Complexity Analysis

Let V be the number of vertices and m be the number of colors.

For every vertex, up to m colors can be tried.

Worst-case Time Complexity

$$O(m^V \times V)$$

The m^V represents the possible color assignments, while checking whether a color is safe may take $O(V)$ using an adjacency matrix.

Auxiliary Space Complexity

$$O(V)$$

because of the color array and recursion stack.

If an adjacency matrix is used, the graph requires:

$$O(V^2)$$

space

11. Complexity Class

The **M-Coloring Decision Problem** is **NP-Complete** for general graphs when $m \geq 3$.

A proposed coloring can be verified in polynomial time by checking every edge and confirming that its endpoints have different colors. However, finding a valid coloring can require exponential time using backtracking in the worst case.

11. Conclusion

The Graph Coloring Problem can be solved using backtracking by assigning colors to vertices one by one. The algorithm checks adjacency constraints before every assignment and backtracks whenever a valid color cannot be

found. Pruning invalid color assignments early reduces unnecessary exploration. The worst-case time complexity is $O(m^V \times V)$ and the auxiliary space complexity is $O(V)$.

Q2 Approximation and Tractability

50 marks

Feedback Vertex Set Problem

Design a pseudocode using Backtracking to remove minimum vertices to eliminate cycles. Clearly identify inputs (graph) and output (vertex set). Develop an algorithm to explore vertex subsets. Use loops and recursion effectively. Apply pruning to avoid unnecessary removals. Ensure pseudocode clarity. Handle all cases.

Q2 – FEEDBACK VERTEX SET PROBLEM

Backtracking Approach

1. Problem Statement

Given a graph $G = (V, E)$, find the **minimum set of vertices** whose removal eliminates all cycles in the graph.

This set is called a **Feedback Vertex Set (FVS)**.

The algorithm uses **Backtracking** to explore different subsets of vertices. For each vertex, it considers two choices:

1. **Include the vertex in the feedback set** and remove it.
2. **Exclude the vertex** and keep it.

The algorithm checks whether the remaining graph contains a cycle. If the graph becomes acyclic, the selected vertices form a valid feedback vertex set. The minimum-sized set is returned.

Assumption: The pseudocode below considers an undirected graph, which is the usual formulation of the Feedback Vertex Set problem.

2. Input and Output

Input

- $G \rightarrow$ Undirected graph with V vertices and E edges.

Output

- FVS $\rightarrow$ Minimum set of vertices whose removal makes the graph acyclic.
- If the graph is already acyclic, the output is an empty set $\{\}$.

3. Pseudocode

ALGORITHM FEEDBACK_VERTEX_SET(G, V)

INPUT:

$G \rightarrow$ Undirected graph with V vertices

OUTPUT:

Minimum Feedback Vertex Set

BEGIN

currentSet $\leftarrow$ empty set

bestSet $\leftarrow$ empty set

bestSize $\leftarrow V + 1$

FVS_BACKTRACK($G, 1$, currentSet)

IF bestSize = $V + 1$ THEN

 PRINT "No Feedback Vertex Set found"

ELSE

 PRINT "Minimum Feedback Vertex Set:"

 PRINT bestSet

 PRINT "Minimum Size:", bestSize

END IF

END

PROCEDURE FVS_BACKTRACK(G , vertex, currentSet)

BEGIN

IF HAS_CYCLE(G) = FALSE THEN

IF size(currentSet) < bestSize THEN

bestSet $\leftarrow$ copy(currentSet)

bestSize $\leftarrow$ size(currentSet)

END IF

RETURN

END IF

IF vertex > V THEN

RETURN

END IF

IF size(currentSet) $\geq$ bestSize THEN

RETURN

END IF

// Choice 1: Include vertex in Feedback Vertex Set

REMOVE vertex FROM G

ADD vertex TO currentSet

FVS_BACKTRACK(G, vertex + 1, currentSet)

ADD vertex BACK TO G

REMOVE vertex FROM currentSet

// Choice 2: Exclude vertex from Feedback Vertex Set

FVS_BACKTRACK(G, vertex + 1, currentSet)

END

4. Cycle Detection Pseudocode

The following function checks whether the current graph contains a cycle.

FUNCTION HAS_CYCLE(G)

BEGIN

visited[1...V] $\leftarrow$ FALSE

FOR vertex $\leftarrow$ 1 TO V DO

IF visited[vertex] = FALSE THEN

IF DFS_CYCLE(vertex, -1) = TRUE THEN

```
        RETURN TRUE
    END IF
```

```
END IF
```

```
END FOR
```

```
RETURN FALSE
```

```
END
```

```
FUNCTION DFS_CYCLE(vertex, parent)
```

```
BEGIN
```

```
    visited[vertex]  $\leftarrow$  TRUE
```

```
    FOR each adjacent vertex u of vertex DO
```

```
        IF visited[u] = FALSE THEN
```

```
            IF DFS_CYCLE(u, vertex) = TRUE THEN
```

```
                RETURN TRUE
```

```
            END IF
```

```
        ELSE IF  $u \neq$  parent THEN
```

```
            RETURN TRUE
```

END IF

END FOR

RETURN FALSE

END

5. Logical Explanation

The algorithm explores different subsets of vertices using backtracking.

Initially:

$\text{currentSet} = \{\}$

The algorithm checks whether the graph contains a cycle.

Case 1 – Graph is acyclic

If:

$\text{HAS_CYCLE}(G) = \text{FALSE}$

then the current selected vertices form a valid Feedback Vertex Set.

The algorithm compares its size with the best solution found so far.

IF $\text{size}(\text{currentSet}) < \text{bestSize}$

the current set becomes the new best solution.

Case 2 – Graph still contains a cycle

The algorithm selects a vertex and makes two choices:

- Include the vertex in the FVS.
- Exclude the vertex from the FVS.

For the include branch, the vertex is temporarily removed from the graph.

After recursively exploring that branch, the vertex is restored. This is the **backtracking step**.

The algorithm then explores the exclude branch.

6. Example

Consider the graph:

A ----- B

| |

| |

D ----- C

This graph contains a cycle:

$A \rightarrow B \rightarrow C \rightarrow D \rightarrow A$

Suppose the algorithm selects vertex A.

Choice 1: Remove A

FVS = {A}

The remaining graph contains:

B ----- C

|

|

D

There is no cycle.

Therefore:

Feedback Vertex Set = {A}

Its size is:

1

The algorithm can continue searching for another solution, but any non-empty cyclic graph needs at least one vertex removed for this example. Therefore {A} is a minimum FVS.

Other valid minimum solutions may be:

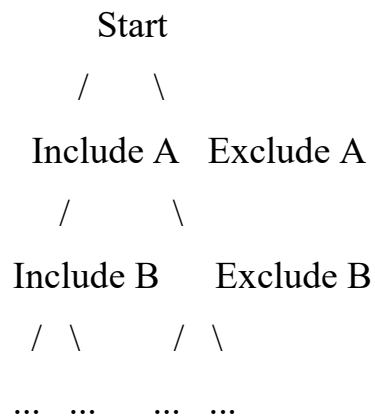
{B}

{C}

{D}

7. State-Space Representation

The state-space tree represents the **include/exclude decision** for each vertex.



At every level:

- **Include** → put the vertex into the Feedback Vertex Set and remove it from the graph.
- **Exclude** → do not put the vertex into the set.

A solution node is reached when the remaining graph is **acyclic**.

8. Recursion Steps

1. Start with an empty feedback vertex set.
2. Check whether the graph contains a cycle.
3. If no cycle exists, save the current set as a valid solution.
4. If a cycle exists, select the next vertex.
5. Include the vertex in the feedback set.
6. Temporarily remove the vertex from the graph.
7. Recursively solve the reduced graph.
8. Restore the vertex and remove it from the current set.
9. Explore the exclude branch.
10. Continue until all possibilities have been considered.
11. Return the smallest valid vertex set found.

9. Pruning Conditions

Backtracking can be improved by eliminating branches that cannot produce a better solution.

Pruning Condition 1 – Graph is already acyclic

IF HAS_CYCLE(G) = FALSE

A valid solution has been obtained, so the branch does not need further exploration.

Pruning Condition 2 – Current solution is already too large

IF size(currentSet) >= bestSize

RETURN

There is no need to explore the branch because it cannot produce a smaller solution.

Pruning Condition 3 – No vertices remain

IF vertex > V

RETURN

There are no further choices to explore.

These pruning conditions reduce unnecessary vertex-subset exploration.

10. Correctness

The algorithm is correct because:

1. It considers both possibilities for every vertex: include or exclude.
2. Therefore, every possible vertex subset can be represented by a path in the state-space tree.
3. Whenever the remaining graph becomes acyclic, the selected vertices form a valid Feedback Vertex Set.
4. The algorithm compares the size of every valid solution with the current best solution.
5. Only solutions smaller than the current best solution are retained.
6. Since all relevant subsets are systematically explored, the smallest valid set is eventually identified.
7. Therefore, the returned set removes all cycles and has minimum possible size.

Hence, the algorithm correctly finds the **minimum Feedback Vertex Set**.

11. Complexity Analysis

Let:

V = Number of vertices

E = Number of edges

For every vertex, there are two possibilities:

Include / Exclude

Therefore, the worst-case number of subsets explored is:

$$O(2^V)$$

For each state, cycle detection using DFS takes:

$$O(V + E)$$

Therefore, the worst-case time complexity is approximately:

$$O(2^V \times (V + E))$$

Auxiliary Space

The recursion and current vertex set require:

$$O(V)$$

additional space, excluding the graph representation.

12. Complexity Class

The **Feedback Vertex Set Decision Problem** is **NP-Complete**.

The decision version asks whether there exists a feedback vertex set of size at most k .

A proposed set can be verified in polynomial time by removing the selected vertices and checking whether the remaining graph is acyclic.

However, finding the minimum feedback vertex set is computationally difficult in general, and the backtracking approach has exponential worst-case complexity.

13. Conclusion

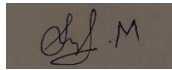
The Feedback Vertex Set Problem can be solved using backtracking by exploring

include/exclude choices for vertices. Whenever the remaining graph becomes acyclic, a valid feedback vertex set is obtained. The algorithm uses pruning to stop branches that cannot improve the current best solution. By systematically exploring the possible vertex subsets, it guarantees that the minimum feedback vertex set is found.

Code of Conduct

I SOUNDARIYA M(192524342) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____



RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Marks Evaluation:

Question No.	Problem Understanding (20%)	Algorithm Design (30%)	Use of Control Structures (20%)	Pseudocode Structure (10%)	Completeness (20%)	Total Marks
Q1 (40 Marks)						
Q2 (30 Marks)						
Q3 (30 Marks)						
Overall Total (100)						

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____

Date: _____	Date: _____	Date: _____
-------------	-------------	-------------